

Exercícios — Aula 12 — Quicksort e particionamento

Técnicas de Programação - 2026/02

Última atualização: July 26, 2026

Instruções

- O centro deste handout é a partição: acompanhe pivô, **menores**, **atual** e trocas.
- Use intervalo fechado [**inicio**, **fim**] quando o pivô está em **fim**.
- Depois da partição, o pivô não participa das chamadas recursivas.

Exercício 1 — Simulação de particionamento

Simule o algoritmo para $v = [8, 3, 7, 2, 5]$, $inicio = 0$, $fim = 4$.

Algorithm 1: Particionar

Result: Executa PARTICIONAR.

Input : array v , int $inicio$, int fim

Output: posição final do pivô

```
pivo ← v[fim]
menores ← inicio
for atual ← inicio to fim − 1 do
    if v[atual] ≤ pivo then
        TROCAR(v, menores, atual)
        menores ← menores + 1
    end
end
TROCAR(v, menores, fim)
return menores
```

atual	v[atual]	menores antes	ação	array após ação	menores depois
0					
1					
2					
3					
fim	pivô		troca final		

Qual é a posição final do pivô? O que fica garantido à esquerda e à direita dele?

Exercício 2 — Simulação de quicksort

Use o particionamento do exercício anterior para simular `QUICKSORT(v, 0, 4)` em `v = [8, 3, 7, 2, 5]`.

Algorithm 2: Quicksort

Result: Executa `QUICKSORT`.

Input : array `v`, int `inicio`, int `fim`

Output: `v` ordenado no próprio lugar

```

if inicio ≥ fim then
    | return
end
p ← PARTICIONAR(v, inicio, fim)
QUICKSORT(v, inicio, p − 1)
QUICKSORT(v, p + 1, fim)

```

Preencha a árvore de chamadas.

```

QUICKSORT(v, 0, 4)
|-- particiona -> p = __, v = __
|-- QUICKSORT(v, __, __)
|   `-- ...
`-- QUICKSORT(v, __, __)
    `-- ...

```

Em quais chamadas o caso base é atingido?

Exercício 3 — Completar TROCAR

Complete o pseudocódigo abaixo.

Algorithm 3: Trocar

Result: Executa TROCAR.

Input : array v , int a , int b

Output: v com as posições a e b trocadas

$temporario \leftarrow$ _____

$v[a] \leftarrow$ _____

$v[b] \leftarrow$ _____

Agora simule:

- $v = [9, 4, 7]$, $a = 0$, $b = 2$;
- $v = [9, 4, 7]$, $a = 1$, $b = 1$.

Por que uma troca com a mesma posição não altera o array?

Exercício 4 — Pior caso com pivô final

Nesta versão, o pivô é sempre o último elemento do intervalo.

Considere $v = [1, 2, 3, 4, 5]$.

1. Qual pivô é escolhido na primeira partição?
2. Qual é a posição final dele?
3. Quais tamanhos têm os dois subproblemas?
4. O que acontece na próxima chamada recursiva?
5. Por que esse padrão leva a custo quadrático?

Repita o raciocínio para $v = [5, 4, 3, 2, 1]$.

Exercício 5 — Valores repetidos

Simule PARTICIONAR para $v = [4, 2, 4, 1, 4]$, com pivô final.

atual	v[atual]	comparação com pivô	troca?	array
0				
1				
2				
3				
fim	pivô		troca final	

Responda:

1. Para qual lado vão os valores iguais ao pivô nesta versão?
2. A posição final do pivô divide bem o array?
3. Por que muitos repetidos podem causar divisões ruins?

Exercício 6 — Quicksort ou mergesort?

Compare os dois algoritmos nos cenários abaixo.

Cenário	Melhor escolha	Justificativa
Preciso de garantia de pior caso $O(n \log n)$ na versão estudada		
Quero usar pouca memória auxiliar além da pilha		
O array pode já estar ordenado e o pivô é sempre o último		
Preciso de estabilidade na versão estudada		
Quero entender ordenação por particionamento		

Depois, escreva uma frase explicando por que quicksort não tem etapa de merge final.

Créditos e reaproveitamento

Exercícios adaptados de simulações de particionamento e formalização de quicksort dos handouts antigos devem indicar:

Adaptado de material de Igor Montagner para a disciplina Técnicas de Programação.

Fonte externa opcional para variações conceituais: Princeton Quicksort, <https://algs4.cs.princeton.edu/23quicksort/>.